UNCUYO
UNIVERSIDAD NACIONAL DE CUYO

FACULTAD DE INGENIERÍA

Ingeniería del Software II

# TRABAJO PRÁCTICO GUÍA N° 2

Profesor:

- Lic. Leandro Spadaro.

Año:

- 2026

**Objetivos**

a) Ejercitar de forma rápida la identificación de clases, atributos, responsabilidades y sus relaciones mediante enunciados cortos.

b) Realizar pequeños ejercicios mediante arquitectura cliente – servidor respetando lo indicado en cada ejercicio junto con sus respectivas pruebas automáticas, utilizando la metodología de desarrollo que el grupo decida (RUP, AGIL, libre, etc), fomentando el espíritu de investigación de los grupos en diferentes tecnologías y arquitecturas basados en el análisis, diseño e implementación orientado a objeto.

c) El conocimiento adquirido en cada uno de los ejercicios pueda ser aplicados a los Trabajos prácticos integradores propuestos para el cursado de la materia.

**Forma de trabajo.**

El estudio y aprendizaje para la resolución de los ejercicios será con metodología autoestudio – autodidacta guiado por el profesor de práctica, donde se establecerá como material de consulta para la resolución de los ejercicios una guía audiovisual conformada por videos almacenados en la plataforma Youtube. Además, los grupos de trabajo podrán utilizar el material de consulta que considere necesario para complementar el conocimiento requerido (libros, videos, inteligencia artificial, etc).

**Equipos de trabajo.**

Los equipos de trabajo serán los conformados al inicio del cursado de la materia.

**Presentación del Trabajo Práctico.**

El grupo deberá presentar los ejercicios codificados en la clase de práctica según el cronograma establecido para tal fin. La codificación deberá ser almacenada en un repositorio público.

Cronograma de presentación:

- Ejercicio N°1: Fecha Evaluación: 09/10/2026 Fecha Entrega: 04/10/2026
- Ejercicio N°2: Fecha Evaluación: 09/10/2026 Fecha Entrega: 04/10/2026
- Ejercicio N°3: Fecha Evaluación: 16/10/2026 Fecha Entrega: 11/10/2026
- Ejercicio N°4: Fecha Evaluación: 16/10/2026 Fecha Entrega: 11/10/2026
- Ejercicio N°5: Fecha Evaluación: 23/10/2026 Fecha Entrega: 25/10/2026
- Ejercicio N°6: Fecha Evaluación: 23/10/2026 Fecha Entrega: 25/10/2026
- Ejercicio N°7: Fecha Evaluación: 30/10/2026 Fecha Entrega: 01/11/2026
- Ejercicio N°8: Fecha Evaluación: 30/10/2026 Fecha Entrega: 01/11/2026

Al finalizar todos los ejercicios se deberá presentar un documento que haga referencia al trabajo práctico donde deberá figurar el enunciado del ejercicio junto con el repositorio que contiene la codificación del mismo.

Cronograma de presentación Individual:

Todos los ejercicios correspondientes a la parte individual de cada ejercicio deberán estar presentados el 08/11/2026.
Respecto de la Evaluación de los ejercicios, esta se realizará en la presentación del grupo.

**Codificación de Ejercicios.**

Para la codificación de los ejercicios, cada grupo podrá utilizar el lenguaje de programación que considere que cumple con las pautas necesarias para cumplir el objetivo solicitado. El lenguaje de referencia que se utilizará durante el cursado es Java, mediante su implementación con el Framework Spring Boot ya que dicho lenguaje fue incorporado en materias cursadas de forma previa, permitiendo que la curva de aprendizaje sea reducida.

**Patrones de programación.**

Los grupos de trabajo deberán aplicar en la resolución de los ejercicios indicado, el conocimiento adquirido en las clases de teoría y práctica, al momento de realizar la programación.

**Organización de los Equipos.**

Los equipos deberán utilizar para la organización de tareas un tablero de tipo Kanban (Tareas pendientes, Tareas en Progreso, Tareas Probadas y Tareas Finalizadas), para cumplir con este objetivo se utilizará la herramienta https://www.atlassian.com/es/software/jira . Los equipos deberán mostrar en las exposiciones como fue desarrollado la división de tareas y la gestión de Historias de Usuario entre los integrantes para lograr el objetivo planteado en los ejercicios.

**Condición de aprobación del trabajo práctico.**

A continuación, se detallan las condiciones de aprobación del trabajo práctico:

1) Presentar el 100% de los ejercicios resueltos antes de la presentación del trabajo integrador N°2.
1) Participación en clases.
2) El 80% de la asistencia individual a clases de práctica de cada miembro del equipo.
3) El 80% de asistencia a clases de consulta de práctica de al menos uno (1) de los integrantes del grupo de trabajo.

# EJERCITACIÓN

Ejercicio N°1: ARQUITECTURA API-REST (PARTE 1).

**PRESENTACIÓN GRUPAL**

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondiente para siguiente problema planteado: Se desea registrar en un sistema para Consultorios Médicos, los turnos de los pacientes. Los pacientes pagan un co-seguro para poder ser atendidos. El turno debe contener el horario en el que asiste el paciente y el médico que lo atenderá.
   Previo a la realización del diagrama de clases, se deberá elaborar las Historias de Usuario que pudieran formar parte de la especificación, junto con sus prototipos y la estimación de tiempos utilizando Planning Poker. Además, se tiene que armar el Product Backlog del proyecto donde se registran las Historias de Usuario, uno de los miembros del equipo debe ejercer el rol de Product Owner para la priorización de las historias de usuario. Buscar al menos 3 Sitios Web del mercado que se dediquen al mismo rubro.

- Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia.

b) Investigación:

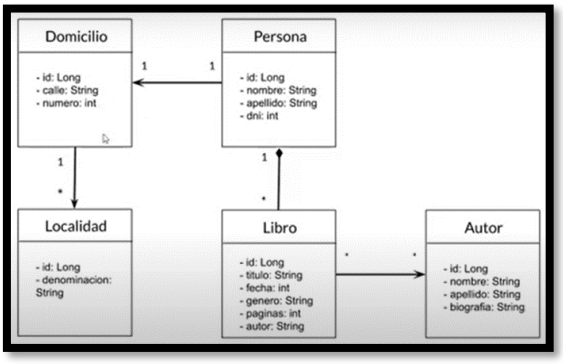
- HTTP:
  - ¿Qué es HTTP y como funciona? ¿Cuáles son los métodos de petición y que realiza cada uno? ¿Qué diferencia existen entre una Petición GET y POST? ¿Cómo es el envió de parámetros, qué significa "=" y que significa "&" cuando se envían parámetros por POST y GET? ¿Cuáles son los códigos de estado en HTTP?

- REST – API:
  - ¿Qué es REST y para que sirve? ¿Qué es una API, para que sirve y cual es el objetivo de combinarlo con REST? ¿Qué es un Endpoint?. Realizar un ejemplo Gráfico del Uso de API REST.
  - Mencione cuales son los patrones de diseño que se aplican para API. Realizar una explicación de cada uno de ellos.

- JSON:
  - ¿Qué es JSON y para qué sirve? ¿Cómo es el formato de JSON? Dar un ejemplo de un objeto y de una colección de Objetos en Java y su equivalente en JSON. ¿Cuáles alternativas existen además de JSON?

- POSTMAN:
  - ¿Qué es Postman? ¿Cuál es la dirección de su sitio oficial? Dar un ejemplo cómo utilizar GET, POST, PUT, DELETE.

c) Dado el siguiente modelo desarrolle el software de tipo web cliente – servidor que permita implementar las relaciones establecidas utilizando arquitectura REST mediante API. El proyecto deberá contar de dos aplicaciones una cliente, que debe utilizar RestTemplate y otro servidor que se comuniquen mediante API REST. La aplicación cliente deberá utilizar DTO's para manipular la vista luego de consumir las API del servidor para mostrarla en un sitio HTML.




d) Agregar al punto anterior la posibilidad que al momento de crear un libro se pueda cargar el archivo PDF correspondiente en el disco C del servidor en la carpeta C://biblioteca//libro_nombrelibro_.pdf. Al momento de consultar el libro se debe poder

abrir en una solapa distinta del sistema el PDF. Refactorizar el diagrama de clases y el código para cumplir con el objetivo.

e) APIS EXTERNAS: Realice un software que consuma información de una API Externa en el BackEnd (utilizar REST Template en el caso de Java), para luego ser visualizada en el FrontEnd.
Utilizar sitios de internet para investigar sobre Apis externas.
Ej.: https://rapidapi.com/ o https://openweathermap.org/api o https://magicloops.dev/es
Utilizar una maqueta gratuita HTML/CSS/Bootstrap para el FrontEnd.
Ej.: https://plantillashtmlgratis.com/categoria-plantillas/bootstrap/
Investigar el acceso la API de WhtasApp y la utilidad que el equipo podría darle en un proyecto futuro.

f) MapStruct: Realizar la guía Audio Visual y crear un proyecto nuevo igual al creado en el punto "C" y utilizar la librería MapStruct para convertir de entidad en DTO y viceversa. La conversión debe ser realizada en las clases de servicio, es decir las clases de servicio reciben DTO's de las clases controladoras y envían DTO´s a las clases controladoras. "NO PUEDEN ENVIARSE ENTIDADES A LAS CLASES CONTROLADORAS" y "NO PUEDEN ENVIARSE DTO'S A LAS CLASES REPOSITORIO".

g) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. Visualizar los primeros videos de la guía "**1) API REST PARTE 1: React con Java (PARTE A)**".

**PRESENTACIÓN INDIVIDUAL**

a) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. "**1) API REST PARTE 1: React con Java (PARTE B)**".

Ejercicio N°2: API REST ((PARTE 2).

**PRESENTACIÓN GRUPAL**

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondientes para siguiente problema planteado. El equipo fue contratado para desarrollar una red social, donde las personar que la utilicen tendrán un usuario para realizar publicaciones. Cada publicación contiene una imagen junto con un comentario. Los usuarios de la red social pueden visualizar las publicaciones de otros usuarios e indicar si les gusto, además de poder realizar comentarios. Los usuarios pueden enviar mensajes de forma privada entre ellos. El comportamiento de la red social es muy similar a la conocida "Instagram". Los usuarios se registran con nombre, apellido, edad, número de documento, teléfono y correo electrónico. Para ingresar a la plataforma utilizarán de usuario el correo electrónico y una contraseña que ellos mismos registrarán.

Se deberá realizar las Historias de Usuario que contemplen las funcionalidades criticas especificadas junto con el prototipado de pantalla, y la estimación de cada historia de usuario utilizando Planning Poker. Investigar que tablero de seguimiento (Scrum / kanban) utilizaría el equipo par el desarrollo del proyecto.

- Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia. Teniendo en cuenta funcionalidades existentes en la actualidad en Instagram.

b) Google Maps: Agregar al ejercicio 1 "c" en la clase dirección para que pueda tener los atributos latitud y longitud de tipo String. Estos atributos deben guardar la información de un punto de referencia de Google Maps. Ej.: -32.88970575178735 (latitud) -68.84457510855037 (longitud) (punto de referencia donde vive cada persona). Luego al mostrar la información de la persona, se deberá modificar el Front para generar un link (botón) que abra en otra solapa del navegador Google Maps con la ubicación. El link se debe formar con https://www.google.com/maps?q=-32.88970575178735,-68.84457510855037

c) Automatización de Tareas: Realizar el ejercicio de la guía audio visual "**2) API REST PARTE 2: Scheduling**". Agregar al código del ejercicio 2 "b" la posibilidad para enviar correos electrónicos 1 día antes de la fecha de vencimiento de devolución del libro, los correos se deben enviar de forma automática. Además, se debe mandar un correo a todas las personas el día de su cumpleaños, con un HTML embebido con salutaciones donde tenga un botón que redireccione a la página de la facultad.

d) Archivos EXCEL y PDF: Realizar el ejercicio de la guía audio visual "**2) API REST PARTE 2: Archivos PDF / Archivos Excel**". Agregar como funcionalidad al ejercicio "c" la posibilidad de descargar en archivo PDF el listado de las personas que realizaron alquiler de libros y en Excel el listado de libros disponibles.

e) Archivos TXT: Utilizando el ejercicio programado en el punto "d", agregar en el menú la opción "Migración", y realiza una funcionalidad que lea de un archivo llamado "migración.txt" el cual posee los datos de los proveedores con sus direcciones para ser ingresados en el sistema. El archivo posee todos sus campos separados por ";", donde los campos son: NOMBRE;APELLIDO;DNI;CALLE;NÚMERO; Realizar el ejercicio de la guía audio visual "**2) API REST PARTE 2: Archivos de Text y tokens**". Ejemplo de la estructura interna del archivo:

Lionel;Escaloneta;293851450;Perú;345;

Pablo;Perez; 293451460;San Martín;1050;

Esteban;Peralta; 291251480;Benavente;121;

f) Realizar el proyecto Tinder de Mascota utilizando API REST. Utilizando RestTemplate junto MapStruct para DTO´s.

g) Investigación Spring IA:

- ¿Qué es Spring IA? ¿Qué ventajas posee? ¿De ejemplos de como se podría aplicar en distintas funcionalidades de sistemas?
- Realizar la Guía Audio Visual "**2) API REST PARTE 2:** Introducción Spring IA (PARTE A)"
- Realizar la Guía Audio Visual "**2) API REST PARTE 2:** Introducción Spring IA (PARTE B)"

h) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. Visualizar los primeros videos de la guía "**2) API REST PARTE 2: React con Java (PARTE A)**".

**PRESENTACIÓN INDIVIDUAL**

a) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. **2) API REST PARTE 2: React con Java (PARTE B)**".

Ejercicio N°3: PATRÓN TEMPLATE METHOD / GENERIC / REFLECTION.

**PRESENTACIÓN GRUPAL**

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondiente para siguiente problema planteado: Trabajamos en un software que permite a los usuarios del mismos realizar el alquiler de cabañas en la provincia de Mendoza, donde cada cabaña puede tener distintos propietarios. Las cabañas tienen capacidad para 2, 4 o 6 personas. Las cabañas se alquilan con un mínimo de 5 días corridos.
Realizar las Historias de Usuario críticas, junto con su estimación de tiempos utilizando Planning Poker y el presupuesto aproximado de costo mediante un documento formal, El equipo podrá utiliza como referencia los valores publicados en el sitio: https://cpcipc.org.ar/honorarios-recomendados/ . Buscar al menos 3 Sitios Web del mercado que se dediquen al mismo rubro.

- Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia.

b) Investigación:

- Patrón Plantilla (Template Method):
  - ¿Qué especifica el patrón plantilla? Realizar el Diagrama de Clases. ¿Cómo se utilizaría el en las clases Controlador y clases de Servicio?

- Generics:

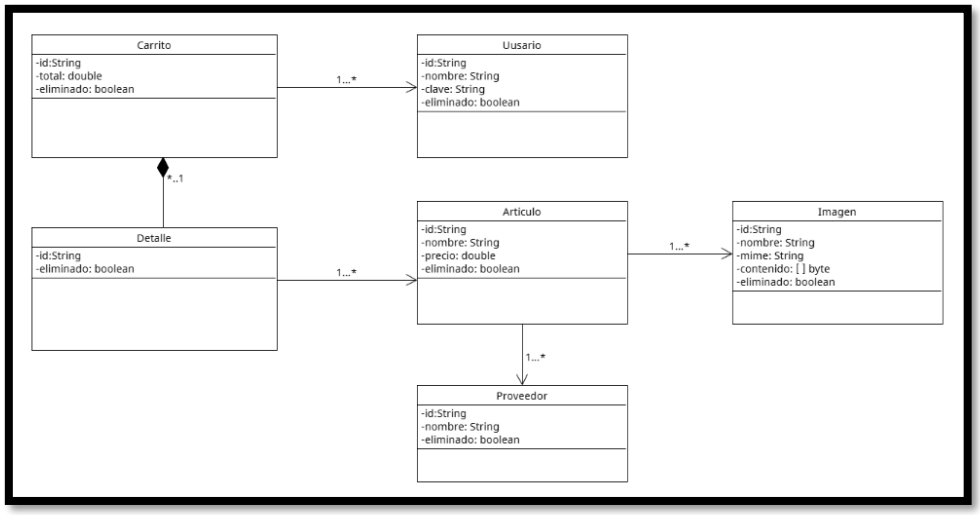
- ¿Qué es Java Generics y cuáles son sus ventajas? ¿Cómo podría complementarse con el patrón plantilla para generalizar las clases Controlador y clases de Servicio?

- Reflection:
  - ¿Qué es Java Reflection y cuáles son sus ventajas? Realizar en código de como se accede a los atributos privados de una clase y como se puede invocar un método.

c) Crear un nuevo proyecto igual al desarrollado en el ejercicio 2 "c", refactorice el código, para aplicar una clase controlador con Template Method y una clase de servicio con Template Method.

d) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. Visualizar los primeros videos de la guía "**3) React con Java (PARTE A)**".

e) Realizar la codificación del siguiente diagrama de clases, aplicando el patrón Template Method, para un carrito de compras con su respectivo FrontEnd y BlackEnd teniendo en cuenta un Template HTML/CSS gratuito para carrito de compras. Antes de comenzar el ejercicio estimar los tiempos de desarrollo con Planning Poker y comparar con los realmente ejecutados. Describir las conclusiones de los tiempos.




f) OpenStreetMap: Investigar y agregar al sistema anterior la dirección de los proveedores donde se guarda la latitud y longitud de cada uno de ellos. El objetivo es mostrar en un mapa la ubicación de todos los proveedores con la librería indicada.

## PRESENTACIÓN INDIVIDUAL

a) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API. Visualizar los primeros videos de la guía "**3) React con Java (PARTE B)**".

Ejercicio N°4: PATRONES GOF: CREACIONALES / ESTRUCTURALES / COMPORTAMIENTO.

**PRESENTACIÓN GRUPAL**

a) Análisis y Diseño:
Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondientes para siguiente problema planteado. Trabajamos en el desarrollo de un software para una agencia de viajes que comercializa paquetes turísticos a medida. La agencia diseña Paquetes Turísticos que combinan Servicios de Transporte (vuelo, autobús), Alojamientos (hoteles, resorts) y Excursiones/Tours. Cada Excursión es coordinada por un Guía de Turismo local. Un Cliente consulta con un Asesor de Viajes, quien confecciona un Presupuesto de Viaje. Una vez confirmado, el presupuesto se convierte en una Reserva de Viaje. La reserva emite Vouchers de Servicio para cada prestador final. Los pagos se estructuran en un Plan de Pagos que registra Cuotas e Historial de Pagos individuales.

   - Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia.

b) Investigación:

   - Patrones GOF (Gang of Four):
     - ¿Cómo se dividen los patrones GOF y que significa cada categoría? Realizar una explicación de cada uno de ellos, con su respectivo diagrama de clases y un ejemplo, destacando cuando es conveniente utilizaros y en que contexto.

c) Crear un proyecto con el código del ejercicio Video Juego (5 “d” práctico guía Parte 1) y adaptarlo para utilizar API REST, aplicando el patrón Template Method.

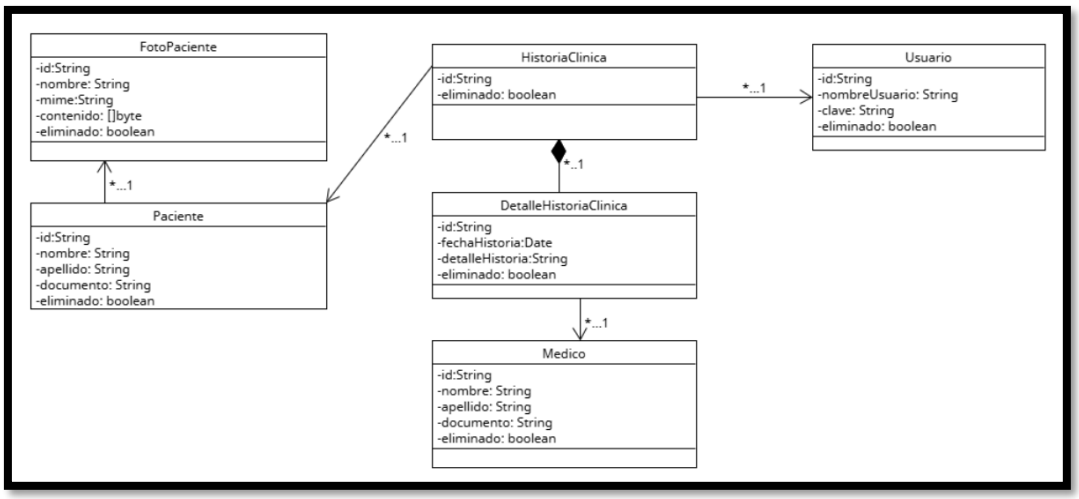
d) Se realiza la refactorización del ejercicio 3 “c” para incorporar patrones Creacionales, Estructurales, Comportamiento y Diseño:

   1. Patrones Creacionales:
      - ✓ Builder: Utilizar el patrón para crear objetos del tipo Libro.
      - ✓ Fábrica: Utilizar el patrón para implementar la creación de Libros Físico y Libros Digitales. Refactorizar el código para que el sistema se adapte a la nueva funcionalidad.
      - ✓ Prototipo: Utilizar el patrón para realizar la clonación de un autor.
   2. Patrones Estructurales:
      - ✓ Adaptador o Librería MapStruct: Utilizar el patrón para adaptar DTO en clase Entidad y viceversa.
   3. Patrones Comportamiento:
      - ✓ Estrategia: Utilizar el patrón para buscar libros por título, género y autor.
      - ✓ Iterador: Utilizar el patrón para recorrer una lista de libros y obtener los que son de un determinado autor.

e) Reportes: Se le solicito al equipo, ver la guía audio visual **“Reportes”** que se realice una mejora para poder descargar en PDF la factura emitida en el sistema del Trabajo

Integrado N°1. El diseño del archivo que se descargue debe ser el similar a una factura de papel. Realizando la Guía Audio Visual el equipo adquiere el conocimiento para realizar la tarea designada. Ej.: formatos de factura: https://invoicehome.com/?locale=es

g) Realizar la codificación del siguiente diagrama de clases con API REST, con su respectivo FrontEnd y BackEnd, aplicando el patrón Template Method, referido a la historia clínica de un paciente que se atiende en un consultorio médico, teniendo en cuenta un Template gratuito. Antes de comenzar el ejercicio estimar los tiempos de desarrollo con Planning Poker y comparar con los realmente ejecutados. Describir las conclusiones de los tiempos.




Luego de realizada la codificación, proponga el uso de patrones creacionales, comportamiento y estructurales (Un patrón por categoría), refactorizando el código. En caso de ser necesario, el equipo, puede agregar nuevas clases o funcionalidades. Se debe fundamentar los motivos de los patrones utilizados explicando el concepto teórico y como se utilizan en el ejercicio.
Realizar 2 versiones del proyecto:

- Utilizando FrontEnd Java con BackEnd Java.
- Utilizando FrontEnd React con BackEnd Java.

**PRESENTACIÓN INDIVIDUAL**

a) Realizar la siguiente Guía Audio Visual sobre Patrones: https://www.youtube.com/watch?v=fzgSFu55Pbw&list=PLQxX2eiEaqbyrYNWCZPBVB32JIGwrAnht

Ejercicio N°5: SEGURIDAD AVANZADO.

**PRESENTACIÓN GRUPAL**

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondiente del siguiente problema planteado: Trabajamos en un sistema que permite publicar noticias de la facultad. En la publicación se muestra una

imagen, su contenido y el autor de la noticia junto con la fecha en la que fue escrita. Utilice el sitio de la facultad como referencia de investigación.
Realizar las Historias de Usuario, el prototipado de pantalla, la estimación de tiempos de las Historias. Realizar el presupuesto aproximado de costo mediante un documento formal, El equipo podrá utiliza como referencia los valores publicados en el sitio: https://cpcipc.org.ar/honorarios-recomendados/ . Buscar al menos 3 Sitios Web del mercado que se dediquen al mismo rubro.
El Equipo debe buscar maquetas html gratuitas que permitan adaptar los requerimientos del cliente con el objetivo de tener una aproximación real de los solicitado. Utilizar una maqueta con información real del sitio de la facultad.
Ej.: https://plantillashtmlgratis.com/categoria-plantillas/educacion/

- Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia.

b) Investigación:

- Certificados de Seguridad SSL (Secure Sockets Layer):
  - ¿Qué es SSL, para que sirve y cómo funciona? ¿Cómo se identifica visualmente un navegador que se está utilizando SSL? Realizar una gráfica del funcionamiento. ¿Cuál es la diferencia entre HTTP y HTTPS? ¿Cómo puede obtener un certificado SSL gratuito?

- Json Web Token (JWT):
  - ¿Qué es JWT (Json Web Token) y cómo funciona? Realizar una gráfica del funcionamiento.

- AUTH0:
  - ¿Qué es AUTH0 y cómo funciona? Realizar una gráfica del funcionamiento.

- API´s Seguras:
  - ¿Cuál es la importancia de que nuestras API's sean seguras? Detallar las buenas prácticas de programación, respecto de seguridad, que se deben tener en cuenta al momento de desarrollar una API.

c) SEGURIDAD JWT 1: Realizar los ejercicios de la Guía Audiovisual JWT (Parte A) y Realizar los ejercicios JWT (Parte B)

d) SEGURIDAD JWT 2: Crear un nuevo proyecto igual al de "Mascotas" de la guía práctica N°1 y aplicar los conocimientos de seguridad adquiridos en la Guía Audio Visual con JWT (Tokens).

e) SEGURIDAD AUTH2: Generar un nuevo proyecto que contenga la misma implementación del ejercicio N°5 "d". Al mismo se le deberá agregar seguridad AUTH2 para cliente (login) y servidor según corresponda.

f) SEGURIDAD API REST: Generar un nuevo proyecto que contenga la misma implementación del ejercicio N°2 “e” y aplique seguridad a las API desarrolladas en el BackEnd, con el método que el equipo crea correcto, para que la misma sea tenida en cuenta al momento de efectuar EndPoint.

g) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API utilizando Seguridad. Visualizar los primeros videos de la guía “**5) React con Java y seguridad (PARTE1)**”.

**PRESENTACIÓN INDIVIDUAL**

a) REACT: Conexión por API Frontend con Backend de distintas tecnologías: Realizar el ejercicio planteado en la Guía Audiovisual donde se utiliza React para el Frotend y Java para el Backend conectándose ambos por API utilizando Seguridad. Visualizar los primeros videos de la guía “**5) React con Java y seguridad (PARTE2)**”.

Ejercicio N°6: INTEGRACIÓN CONTINUA Y DESPLIEGUE.

**PRESENTACIÓN GRUPAL**

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondiente del siguiente problema planteado: El equipo trabaja en el desarrollo de software para una cadena de restaurante (sucursal) donde cada restaurant posee una caja propia donde se registran los movimientos de ingreso y salida de dinero cada vez que un comensal realiza el pago de lo consumido.
Teniendo en cuenta que el Equipo accede a información confidencial, el cliente desea que se firme un acuerdo de confidencialidad. Realizar las Historias de Usuario, el prototipado de pantalla, la estimación de tiempos de las Historias. Realizar el presupuesto aproximado de costo mediante un documento formal, El equipo podrá utiliza como referencia los valores publicados en el sitio: https://cpcipc.org.ar/honorarios-recomendados/ . Buscar al menos 3 Sitios Web del mercado que se dediquen al mismo rubro.

b) Investigación:

- DevOps:
  - ¿Qué significa DevOps, para que sirve y cómo funciona? ¿Qué relación tiene con las metodologías Ágiles y cuales son sus diferencias? Realizar un gráfico de DevOps mostrando las herramientas que se sugieren utilizar explicando cada una de ellas.

- Ambiente de Producción y Despliegue en Producción:
  - ¿Qué significa que una aplicación se despliega en un entorno de Producción? ¿Cuál es la diferencia en el desarrollo de software de un entorno de desarrollo, entorno de pruebas y entorno de producción?

- ¿Qué es un dominio de internet, para qué sirve y cómo funciona? Dar 5 ejemplos de dominios conocidos ¿Qué es un servidor DNS (Sistema de Nombre de Dominio)? ¿Cómo se pueden obtener un dominio en Argentina y en el resto del mundo? ¿Cómo puede obtenerse un dominio gratuito?

- ¿Qué es un servidor web? ¿Qué significa poder hacer Hosting de una aplicación? ¿Qué características y costos tiene DON WEB para aplicaciones de tipo SpringBoot? ¿Qué servidores web internacionales existen para realizar Hosting?

- ¿Qué es DOCKER y para qué sirve? Realizar me manera Gráfica de funcionamiento para una aplicación SpringBoot. ¿Qué es KUBERNET y para qué sirve? Realizar me manera Gráfica de funcionamiento para una aplicación SpringBoot. Mencionar las diferencias entre DOCKER y KUBERNET.

- Integración Continua y Despliegue Continuo:
  - ¿Qué significa Integración Continua (CI) y que significa despliegue continuo (CD)?, ¿Cuáles son sus ventajas? Realizar me manera Gráfica el proceso de CI/CD en el desarrollo de una aplicación.

c) Automatización de despliegue: Utilizando una herramienta de automatización de despliegue tipo DOCKER el equipo prepara la aplicación para despliegue en Producción. Visualizar los primeros videos de la guía "**7) Docker y Deploy**".

d) Teniendo en cuenta la aplicación del Trabajo Practico Integrador N°1 con Seguridad, deberá implementar la puesta en marcha en producción en un servidor real. Para cumplir con el objetivo utilizará los servicios de despliegue ofrecidos por la empresa de hosting "Don Web" https://donweb.com/es-ar/ . Visualizar el 3er. video de la guía "**6) Docker y Deploy**".

e) Integración Continua (CI/CD):

- 1) Trabajo Práctico Integrador con Seguridad, para practicar el uso de servidor de integración continua, por ejemplo, para la automatización de despliegue o test unitarios. Visualizar el 4to. video de la guía "**7) Jenkins**".
- 2) Realizar el ejercicio que se describe en la Guía Audiovisual "**7) CI/CD (Parte A)**".
- 3) Realizar el ejercicio que se describe en la Guía Audiovisual "**7) CI/CD (Parte B)**".

Ejercicio N°7: ARQUITECTURA MICROSERVICIOS / CLOUD.

**PRESENTACIÓN GRUPAL**

a) Investigación:
   - ¿Qué es la arquitectura basada en Microservicios? ¿cuál es la diferencia con la arquitectura cliente servidor monolítica? ¿Cuáles son sus ventajas y desventajas? ¿Cuáles propuestas comerciales conocidas de implementación para microservicios existen (Pagas/Open Source) en el mercado?, realizar

un cuadro comparativo. Describir de forma gráfica el funcionamiento de una aplicación basada en microservicios.
- ¿Qué es un Gateway en la tecnología de microservicios y para qué se utiliza?
- ¿Qué es un balanceador de carga y para qué sirve? ¿Qué relación tiene un balanceador de carga con la arquitectura de microservicios?

b) Introducción Microservicios: Teniendo en cuenta el contenido teórico/práctico visto en clase, realizar el ejercicio identificado en el anexo "GUÍA AUDIOVISUAL" con el título "**7) Introducción MICROSERVICIOS**".

c) Investigación:
- ¿Qué es Spring Cloud? ¿Cuáles son sus ventajas? ¿Qué mejoras aplica para la programación de microservicios?

d) Introducción Cloud: Teniendo en cuenta el contenido teórico/práctico visto en clase, realizar el ejercicio identificado en el anexo "GUÍA AUDIOVISUAL" con el título "**7) Introducción CLOUD**".

Ejercicio N°8: ARQUITECTURAS LIMPIAS.

**PRESENTACIÓN GRUPAL**

a) Investigación:
- ¿Qué significa que una arquitectura sea limpia "clean architecture"? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencia existe entre "clean architecture" y la arquitectura por capas tradicional? Mostrar con un gráfico la diferencia entre ambas.
- ¿Qué es la arquitectura Hexagonal? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencias existen entre "clean architecture"?

b) Introducción: Realizar el ejercicio propuesto en el anexo "GUÍA AUDIOVISUAL" con el título "**08) ARQUITECTURAS LIMPIAS**".

**Mesas Examen Final.**

Los alumnos que al finalizar el cursado de la materia, su condición fuera de alumno regular o en condición de alumno libre si la materia permitiera acceder a la instancia mencionada, deberán concurrir el día de la evaluación con el trabajo práctico cumpliendo las siguientes condiciones:

A) Alumnos Regulares y Alumnos Libres: Trabajo Práctico Completo e individual según lo especificado en cada punto.
B) Alumnos Libres: Se deberá presentar cada sección "a)" de los ejercicios codificados teniendo en cuenta el Patrón MVC, arquitectura API-REST y aspectos Seguridad en el desarrollo. Se deberá contemplar utilizar patrones de diseño de software creacionales, estructurales y de comportamiento en cada uno de los ejercicios según el criterio del alumno, argumentado lo motivos de la implementación de los mismos.

# GUÍA AUDIOVISUAL DE REFERENCIA

## 1) API REST PARTE 1

Conceptos Teóricos:

https://www.youtube.com/watch?si=a2cutnilG4rW__BP&v=RhxOTqFbI5Q&feature=youtu.be

https://www.youtube.com/watch?v=yLG2pM5r3xs&list=PLgwlfcqa5h3xj-GyibK7yUPseOvl9puFm

Desarrollo Ejercicio:

- API REST

https://www.youtube.com/watch?v=LGck-So93TA&list=PLRFOqDrY-6nvhgKJ2Db1h8aPJm3zaX6eO

- Subir Archivo en el Servidor

https://www.youtube.com/watch?v=h5YW-_xgmlo

- API EXTERNA

https://www.youtube.com/watch?v=LaJtCXROTvo

https://www.youtube.com/watch?v=oJ7-Ou9LcQ8

- MAPSTRUCT

https://www.youtube.com/watch?v=6GWtSKQoliQ&t=316s

https://www.youtube.com/watch?v=gBaCpsqsWfY

https://www.youtube.com/watch?v=nMIdoGcE5XE

- React con Java (PARTE A)

https://www.youtube.com/watch?v=Vxxkt7SCdGY

- React con Java (PARTE B)

https://www.youtube.com/watch?v=42XW4hwhjHA

Ejercicio Año 2024:

https://github.com/Coppede-Santos/Ing_software_2_tp_3

https://github.com/JoacoR7/IS2_TP6_APIREST_CONSUMER

https://github.com/aidalaricchia2/ApiRestConsumer y https://github.com/aidalaricchia2/Api-Rest

## 2) API REST PARTE 2

Desarrollo Ejercicio.

- Scheduling

https://www.youtube.com/watch?v=nQBYCytWhJY

- Archivos PDF

https://www.youtube.com/watch?v=_FUCwZ8Hf8w

- Archivos Excel

https://www.youtube.com/watch?v=U_CDd796Lz0

https://www.youtube.com/watch?v=azHQKsmrTiI

- Archivos de Text y tokens

https://www.youtube.com/watch?v=uqZEOO5MU_M&t=9s

https://www.youtube.com/watch?v=ttgV5kfiH7Y

- Spring IA (PARTE A)

https://www.youtube.com/watch?v=sQwpfrBVYEU

- Spring IA (PARTE B)

https://www.youtube.com/watch?v=y1PSwxntY6A

- React con Java (PARTE A)

https://www.youtube.com/watch?v=tlurHSdY6-M

- React con Java (PARTE B)

https://www.youtube.com/watch?v=L5yihOAQOyI

## 3 y 4) PATRONES DE DISEÑO.

Conceptos Teóricos:

https://refactoring.guru/es/design-patterns

https://www.youtube.com/watch?v=x-k6gDI5ZqA&list=PL4bT56Uw3S4x1_IhP0kQZ1mszuLvoNMJF

https://medium.com/@abhishekranjandev/building-your-own-geolocation-platform-like-google-maps-using-openstreetmap-and-java-spring-boot-ec4ea6c77338

https://medium.com/@hermanmaleiane/spring-boot-thymeleaf-leaflet-js-mapping-corona-virus-a8309c5a0b6d

Desarrollo Ejercicio:

- Reportes

https://www.youtube.com/watch?v=pX7K0M7CTMU

https://www.youtube.com/watch?v=Gvwa-_eNW5Y

https://www.youtube.com/watch?v=VxbWjeHXhTY

- React con Java (PARTE A)

https://www.youtube.com/watch?v=p8tBD3YTCPo

- React con Java (PARTE B)

https://www.youtube.com/watch?v=LuM5c_f6nUE

## 5) Seguridad Avanzado: Parte 2 (Spring Security / OAUT2 / JWT)

Conceptos Teóricos:

Roles, Tokens, Autenticación y Autorización:

1. https://www.youtube.com/watch?v=X1CIHh0XzkU
2. https://www.youtube.com/watch?v=0AkGyPpJgaU
3. https://www.youtube.com/watch?v=f4Q_naeVliA

JWT:

4. https://www.youtube.com/watch?v=P8i2ily6tyE
5. https://www.youtube.com/watch?v=capnr5ULxGQ

OAuth2:

6. https://docs.spring.io/spring-security/reference/servlet/oauth2/index.html
7. https://www.geeksforgeeks.org/advance-java/spring-boot-oauth2-authentication-and-authorization/

Desarrollo Ejercicio:

8. JWT (Ejercicio A)

   https://www.youtube.com/watch?v=nwqQYCM4YT8

9. JWT (Ejercicio B)

   https://www.youtube.com/watch?v=5990oFzJUa4

10. OAuth2

https://www.youtube.com/watch?v=TrjDM6_DuQw

https://www.youtube.com/watch?v=zDzvuTBrP1I&list=PLb9gJ6hvvBcdon6FkVS5oavn6vQy134zv

11. React y Java con seguridad (PARTE1)
https://www.youtube.com/watch?v=ANxXtaGnmsQ&list=PLRFOqDrY-6ntsecxyXaV7y8CWgBGkIq2C

12. React y Java con seguridad (PARTE2)

https://www.youtube.com/playlist?list=PL9l1zUfnZkZnmca4gYPVQQ9msSR5tQTdk

Ejercicio Año 2024:

https://github.com/paulisuden/SeguridadOAuth2

https://github.com/MauroSorbello/IngSoft2/tree/master/tp7/biblioteca/biblioteca

https://github.com/adrzanbar/rest-api-security

# 6) DESPLIEGUE.

Desarrollo Ejercicio:

1. DOCKER

https://www.youtube.com/watch?v=OEvHEeDpg_o

https://www.youtube.com/watch?v=cWuirzMOwwg

2. DEPLOY

https://www.youtube.com/watch?v=cMXTd6PoFpo

3. JENKINS

https://www.youtube.com/playlist?list=PLRFOqDrY-6ns9LOJn1XU0oOICnu8up26x

4. CI/CD

Parte A: https://www.youtube.com/watch?v=4YFQno2cGRQ

Parte B: https://www.youtube.com/watch?v=SdZENFSosiQ

# 7) MICROSERVICIOS / CLOUD.

1. Introducción MICROSERVICIOS.

https://www.youtube.com/watch?v=rC5ES1vmRmc&list=PLx89vzy-Ta0pvP5yEdr4KPePSQREJywa4

2. Introducción Cloud.

https://www.youtube.com/watch?v=JJc1b5-mZDY&list=PL0W7WdoXu6O9VWgpDY10vEWt7G6xV9XTE

## 8) ARQUITECTURAS LIMPIAS.

1. Introducción Arquitectura Clean.

https://www.youtube.com/watch?v=GDXrdZlzdHg

2. Introducción Arquitectura Hexagonal.

https://www.youtube.com/watch?v=yE_0DGyWPv0